

CSE152B - Spring 2022: Homework 3

Q1: Using SphereFace [3] for Face Verification

1. In the first section, we will test a pretrained model of SphereFace on LFW [4] dataset.
The [LFW dataset](#) is on `~/public/lfw`. The dataset contains 6000 pairs of human face images with ground truth labels for whether they are from the same identity.
2. The PyTorch code of SphereFace is located in `./sphereFace`, which is modified based on the open source code from https://github.com/clcarwin/sphereface_pytorch.
3. Run the following commands and report the accuracy of SphereFace on LFW verification. **(5 points)**

```
cd sphereFace
tar -zxvf model.tar.gz
python lfw_eval.py --model ./model/sphere20a_20171020.pth --net
faceNet --lfw ~/public/lfw/lfw-deepfunneled/lfw-deepfunneled/
```

LFWACC=0.9610 std=0.0066 thd=0.2450

4. Explain briefly how the following steps are performed when evaluating on LFW dataset:
 - A. Given the features extracted from the network, what is the metric used to measure the distance between two faces? (`lfw_eval.py` : Line 135) **(5 points)**
 - B. How is the threshold set to determine whether two faces are from the same identity? How is the accuracy computed? (`lfw_eval.py` : Line 141 to 148) **(10 points)**

A. They measure the distance using the cosine of the two faces (the dot product divided by the norms) B. The threshold is set by taking the mean of the "best thresholds", which were determined by the `find_best_threshold` function (takes in thresholds and finds threshold with best accuracy)

The accuracy is computed by taking the passed in prediction and comparing the distance of the prediction and the real point to the threshold, if the distance is smaller, it then marks a true for that point, then it compares the number of true points (1s) to the number of false points (0s).

In []:

5. An important step before face recognition is face alignment, in which we warp and crop the image based on the location of facial landmarks.

A. Briefly describe how we warp and crop the image. (`lfw_eval.py` : Line 11-26) **(5 points)**

B. Instead of doing face alignment, crop an image patch of height 112 pixels and width 96 pixels at the center of the image. Report the accuracy. **(10 points)**

A. we set reference points for where we want the landmarks to be and we reshape and recast the given arrays for compadibility with the imported function. Then the imported function (with a long name) does the landmark-based alignment with a similarity transform then returns it in a format that it better suited for the transform which crops and returns the image of the specified size with features at the specified locations. B.LFWACC=0.8847
std=0.0155 thd=0.3210

Q2: Using MTCNN [5] for Detecting Face Landmarks

1. Instead of using provided facial landmarks, we will now use MTCNN [5] for detecting them. The code is located at `./mtcnn`. Run the following commands to generate the facial landmarks. Include two example outputs in your report. **(5 points)**

```
cd mtcnn
python lfw_landmark.py --lfw ~/public/lfw/lfw-deepfunneled/lfw-deepfunneled/
```

```
Adam_Sandler/Adam_Sandler_0004.jpg 106 111 143 109 134 137 110 156 143 154
Adam_Sandler/Adam_Sandler_0003.jpg 106 114 144 113 132 134 111 162 142 162
Adam_Sandler/Adam_Sandler_0002.jpg 107 117 146 110 118 143 116 170 149 168
Adam_Sandler/Adam_Sandler_0001.jpg 105 114 145 111 128 144 117 167 142 166
Adam_Scott/Adam_Scott_0002.jpg 102 118 146 110 124 132 112 166 150 160
Adam_Scott/Adam_Scott_0001.jpg 105 106 144 108 124 138 113 153 146 152
```

2. Go to the `sphereFace` directory, run the following commands by setting flag `alignmentMode` to be 2. Report the error using the predicted facial landmarks. **(5 points)**

```
cd sphereFace
python lfw_eval.py --model ./model/sphere20a_20171020.pth --net
faceNet --lfw ~/public/lfw/lfw-deepfunneled/lfw-deepfunneled/ --
alignmentMode 2
```

LFWACC=0.9840 std=0.0045 thd=0.3000

3. Next, answer the following questions:

A. Is the result better than using the landmarks provided in the previous question? If not, how can you improve performance? **(5 points)**

- B. What are the steps adopted by the method to achieve real-time speed? **(5 points)**
- C. Briefly describe how non-maximal suppression (NMS) is implemented in this method. (`src/box_utils.py` : Line 5-68) **(5 points)**

A. The result is better than using the landmarks provided in the previous question, as the accuracy rose almost 10 percent, wow.

B. The method uses efficient modes of calculations like a pretrained network and compadibility with GPUs for vector operations, which boost the speed of the functon.

C. In the method, non-maximal suppression is implemented through finding the position of the data points and creating boxes around them to check overlap with other points. If that overlap is too large, then the points are deleted to ensure non-maximal suppression.

Q3: Training CosFace [7] on CASIA Dataset [6]

In this section, you are required to implement CosFace based on the code you use in the previous section, train it on CASIA dataset and test on LFW dataset.

1. You will now train CosFace [7] on the CASIA dataset [6] and test on the LFW dataset [4]. In this section, the skeleton code for training is given. CASIA-Webface dataset can be found at `~/public/CASIA-WebFace` .

Go to directory `./cosFace` and open `faceNet.py` . Under class `CustomLinear` implement function $\psi(\theta_{y_i}, i) = \cos(\theta_{y_i}, i) - m$ which is the cosine distance with margin m . Under class `CustomLoss` , implement the loss function with the returned $\psi(\theta_{y_i}, i)$ and $\cos(\theta_j, i)$. You may check (and duly cite) any open source implementation for hints on improving the performance. **(15 points)**

```
# IMPLEMENT phi_theta
phi_theta = cos_theta - self.m

class CustomLoss(nn.Module):
    def __init__(self, s=64 ):
        super(CustomLoss, self).__init__()
        self.s = s

    def forward(self, input, target):
        cos_theta, phi_theta = input
        target = target.view(-1,1) #size=(B,1)

        # IMPLEMENT loss
        one_hot = torch.zeros_like(cos_theta)
        one_hot.scatter_(1, target, 1)
        output = torch.where(one_hot.bool(), phi_theta, cos_theta)
        output = output * self.s
        loss = F.cross_entropy(output, target.squeeze())
```

```

_, predictedLabel = torch.max(cos_theta.data, 1)
predictedLabel = predictedLabel.view(-1, 1)
accuracy =
(predictedLabel.eq(target.data).cpu().sum().item() ) /
float(target.size(0) )

return loss, accuracy

```

2. The original architecture of FaceNet in `cosFace/FaceNet.py` is a 20-layer residual network as described in Table 2 of [3], but without batch normalization. Now add batch normalization after every convolutional and fully connected layer. Train the new network on CASIA dataset and test on LFW dataset. You are free to change any hyper parameters but report the hyper parameters that you think might influence the performance. Following is a demonstration of a residual block with 128 filters and kernel size 3×3 :

$$\begin{aligned}
 y &= \text{CONV}_{3 \times 3, 128}(x) & (1) \\
 y &= \text{BatchNorm}(y) & (2) \\
 y &= \text{PReLU}(y) & (3) \\
 y &= \text{CONV}_{3 \times 3, 128}(y) & (4) \\
 y &= \text{BatchNorm}(y) & (5) \\
 y &= \text{PReLU}(y) & (6) \\
 \text{OUT} &= x + y & (7)
 \end{aligned}$$

- Draw the training curves for accuracy and loss on CASIA and compare to the curve without batch normalization. **(10 points)**
- Report accuracy on the LFW dataset, evaluated using `lfw_eval.py`. **(10 points)**
- Do you achieve better performance on LFW? If yes, explain how batch normalization helps. If not, try to explain why the results are worse. **(10 points)**

Answer here.

```

In [1]: import numpy as np
import matplotlib.pyplot as plt

accuracy = np.load('./cosFace/checkpoint/accuracy.npy')
loss = np.load('./cosFace/checkpoint/loss.npy')

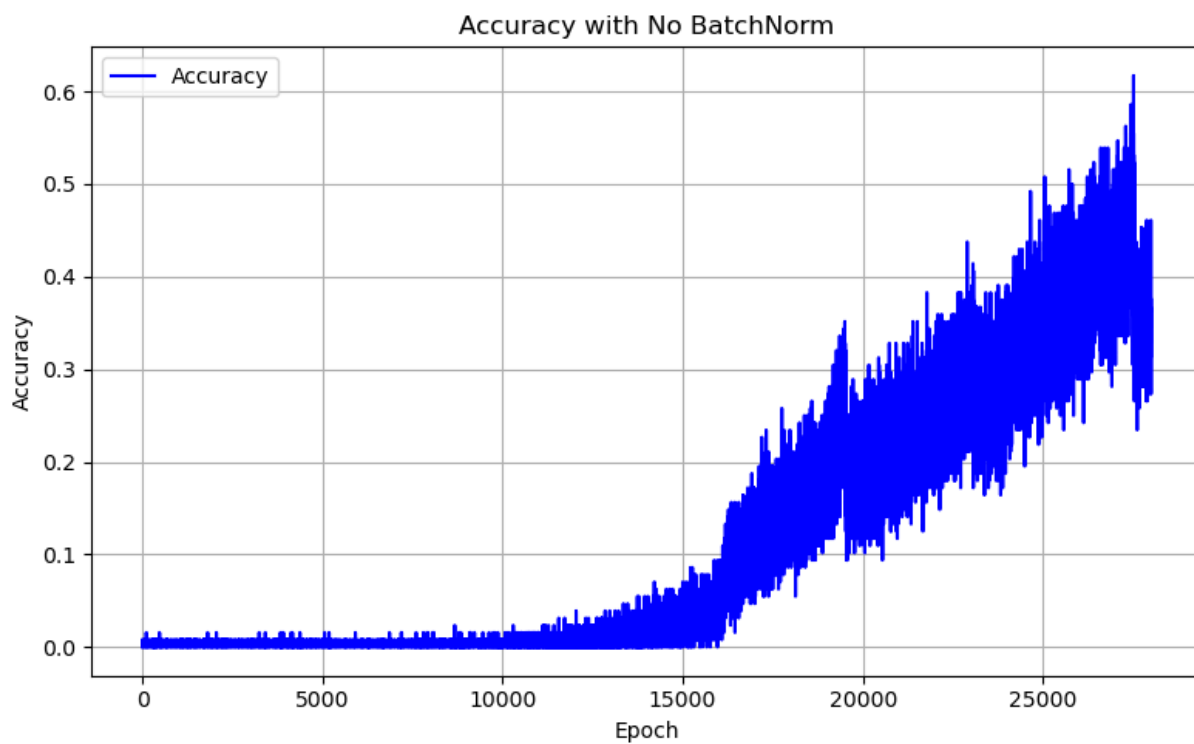
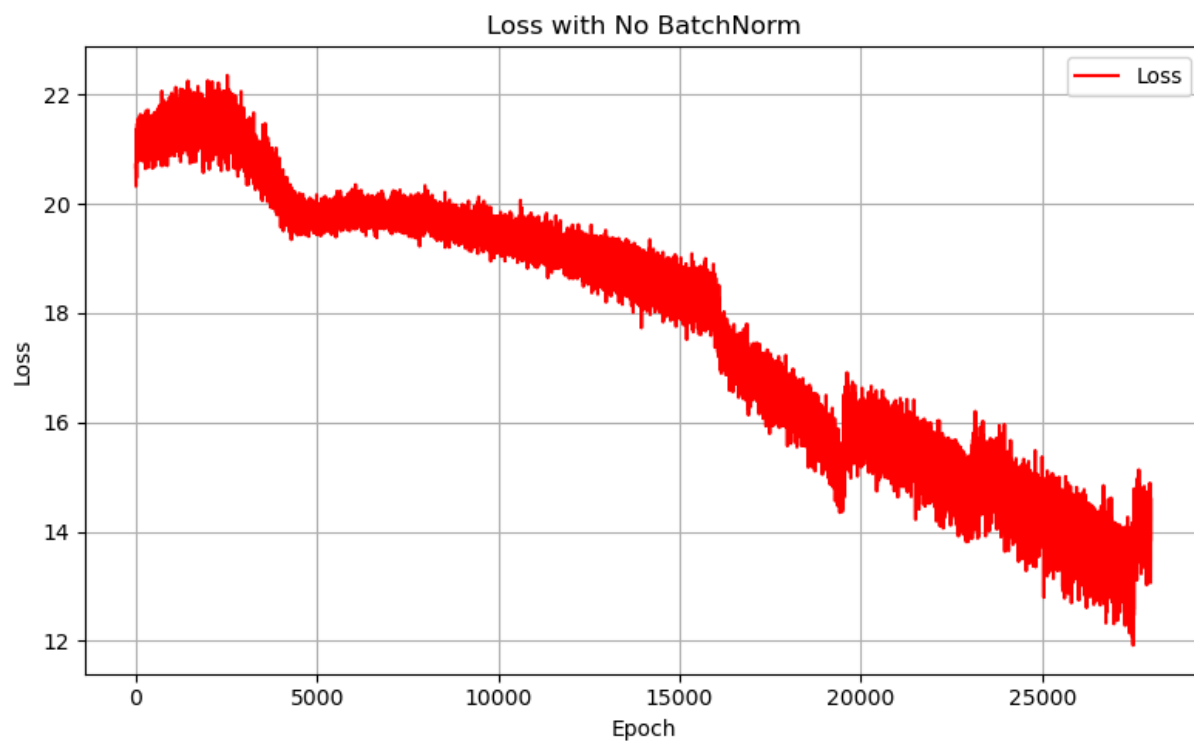
plt.figure(figsize=(8, 5))
plt.plot(loss, label='Loss', color='red')
plt.title('Loss with No BatchNorm')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

# Create figure for Accuracy
plt.figure(figsize=(8, 5))

```



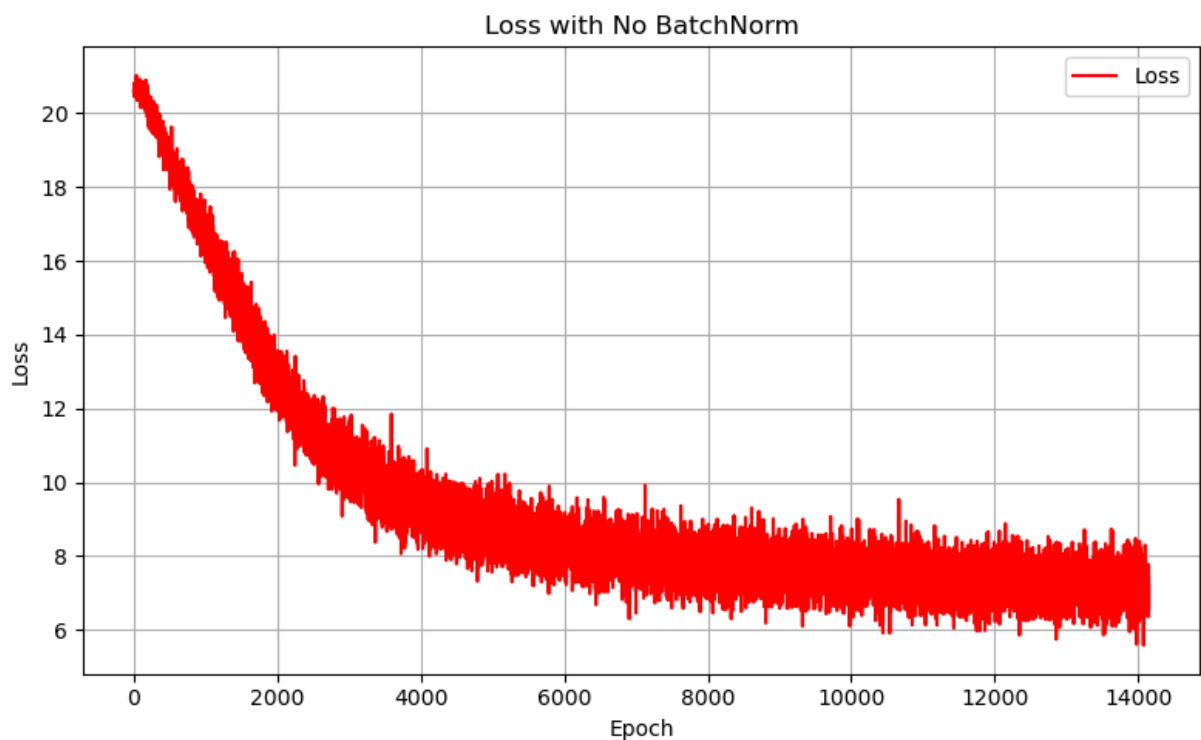
```
plt.plot(accuracy, label='Accuracy', color='blue')
plt.title('Accuracy with No BatchNorm')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

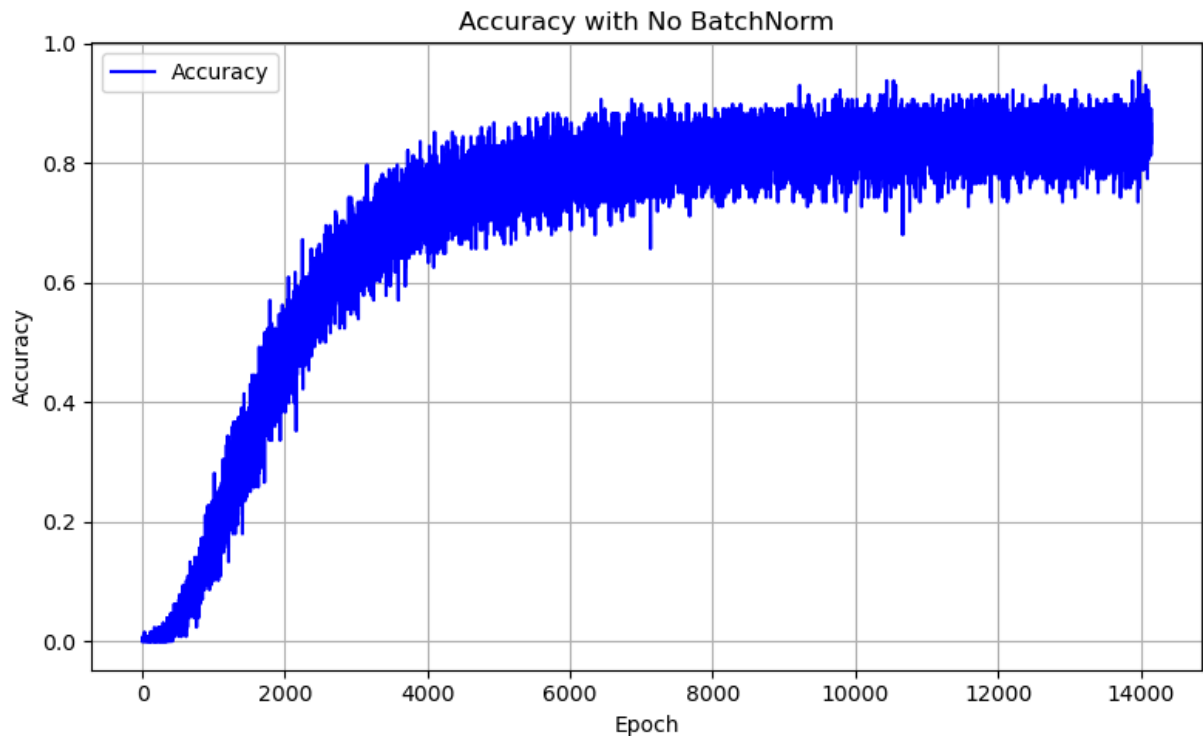


```
In [3]: import numpy as np
import matplotlib.pyplot as plt
accuracy = np.load('./cosFace/daBatchNorm/accuracy.npy')
loss = np.load('./cosFace/daBatchNorm/loss.npy')

plt.figure(figsize=(8, 5))
plt.plot(loss, label='Loss', color='red')
plt.title('Loss with No BatchNorm')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(accuracy, label='Accuracy', color='blue')
plt.title('Accuracy with No BatchNorm')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```





The LFW Eval:

LFWACC=0.8668 std=0.0161 thd=0.3400

It did not perform better than Q1 but perhaps with more training it could. The accuracy was much higher with the BatchNorm than without, due to the efficient normalization to deal with skews and help keep computations normalized, smoother, and more stable optimization, and prevents overfitting.

3. If you achieve better performance compared to SphereFace in Q1, well done! Can you provide a reason? If you do not outperform SphereFace, can you provide a cause? **(10 points)**

I did not achieve better performance, one reason could be the training time and iterations were far fewer for the CosFace network.

4. Randomly choose 10 identities from the CASIA dataset, forward pass all their images through the CosFace network and visualize the normalized features (embeddings) using tSNE [8]. You can use code from <https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html> for visualization. Try a few random samples and include the figure that you consider most illustrative of the method. **(10 points)**

```
In [7]: plt.figure(figsize=(8, 6))
img = plt.imread('cosFace/tsne_results/tsne_face_embeddings_seed42.png')
plt.imshow(img)
```

```
plt.axis('off')
plt.show()
```



Q4: Human Pose Estimation with Convolutional Pose Machines (CPM) [1]

In this question, you will be given code adapted from [PyTorch implementation of one of the CPM model \[1\]](#), which is an follow-up work of the original Convolution Pose Machine [2], and shares the Part Confidence Maps estimation module with [2]. In this question you will be given a trained CPM model and gain insights about the model design and outputs.

- `cd pytorch-openpose`
- `ln -s ~/public/pytorch-openpose model`

1. Run the inference code on the given image.

(a) Visualize the output keypoint detection result. **(5 points)**

```
In [2]: import cv2
import matplotlib.pyplot as plt
import copy
```

```

import numpy as np
import sys
sys.path.insert(0, 'pytorch-openpose')

from src import model
from src import util
from src.body import Body
from src.hand import Hand

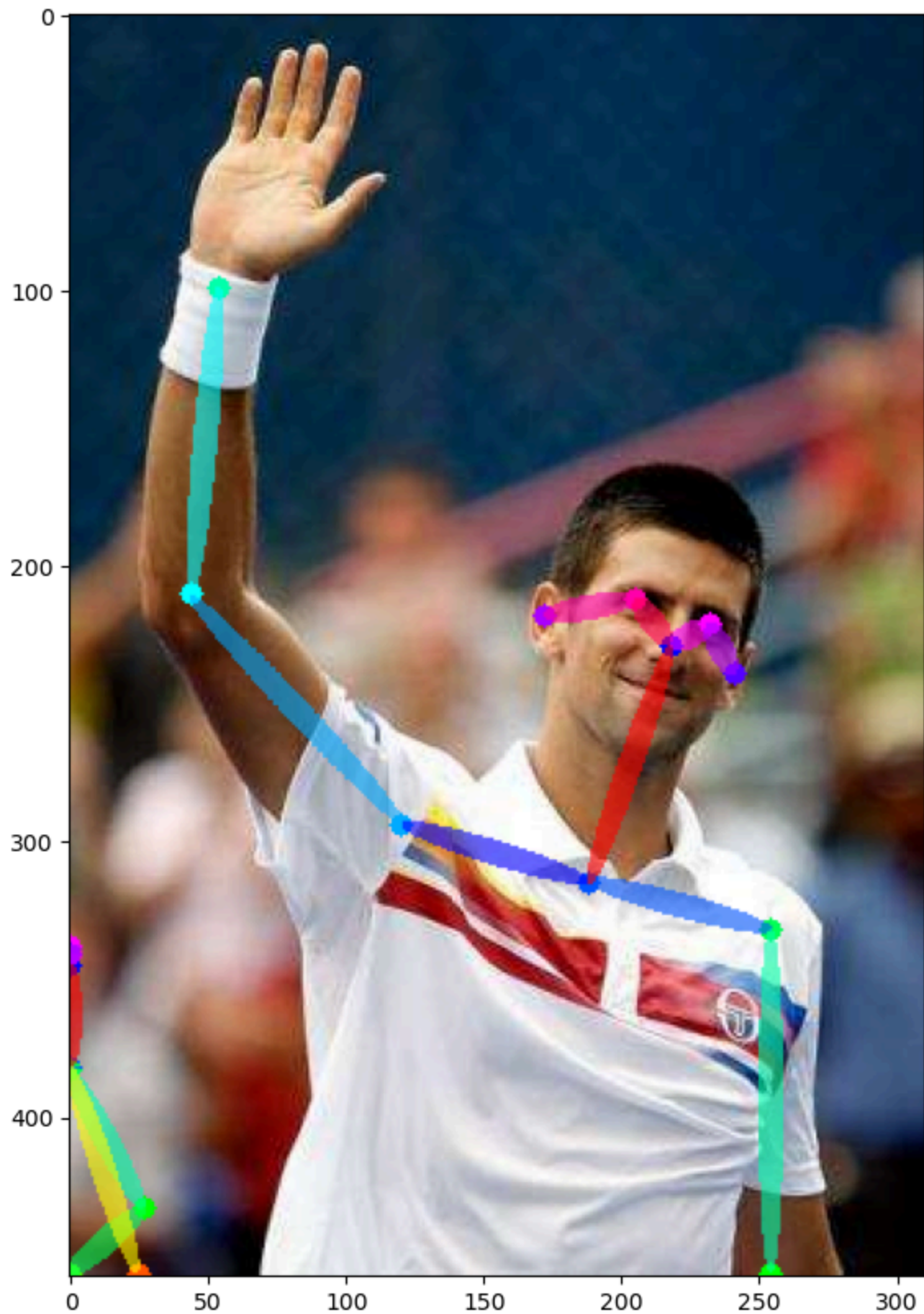
body_estimation = Body('pytorch-openpose/model/body_pose_model.pth')

test_image = 'pytorch-openpose/images/demo.jpg'
oriImg = cv2.imread(test_image) # B,G,R order
candidate, subset, heatmap_list, heatmap_list_converted_list = body_estimation(oriI
heatmap_0 = heatmap_list[0]
print(f"Heatmap shape: {heatmap_0.shape}")
canvas = copy.deepcopy(oriImg)
canvas = util.draw_bodypose(canvas, candidate, subset)

plt.figure(figsize=(10, 10))
plt.imshow(canvas[:, :,
                [2, 1, 0]])
# plt.axis('off')
plt.show()

```

Original image shape: (458, 310, 3), New image shape: (1, 3, 184, 128)
Heatmap shape: torch.Size([1, 19, 23, 16])



(b) What is the ratio of size $\lambda = H/H'$ between the input image **im** (Line 57 of `body.py`) of shape $[1, 3, H, W]$ and the output heatmap **heatmap_0** of shape $[1, D, H', W']$? (5 points)

$$184/23 = 8$$

(c) What module in the model is responsible for this scaling? (5 points)

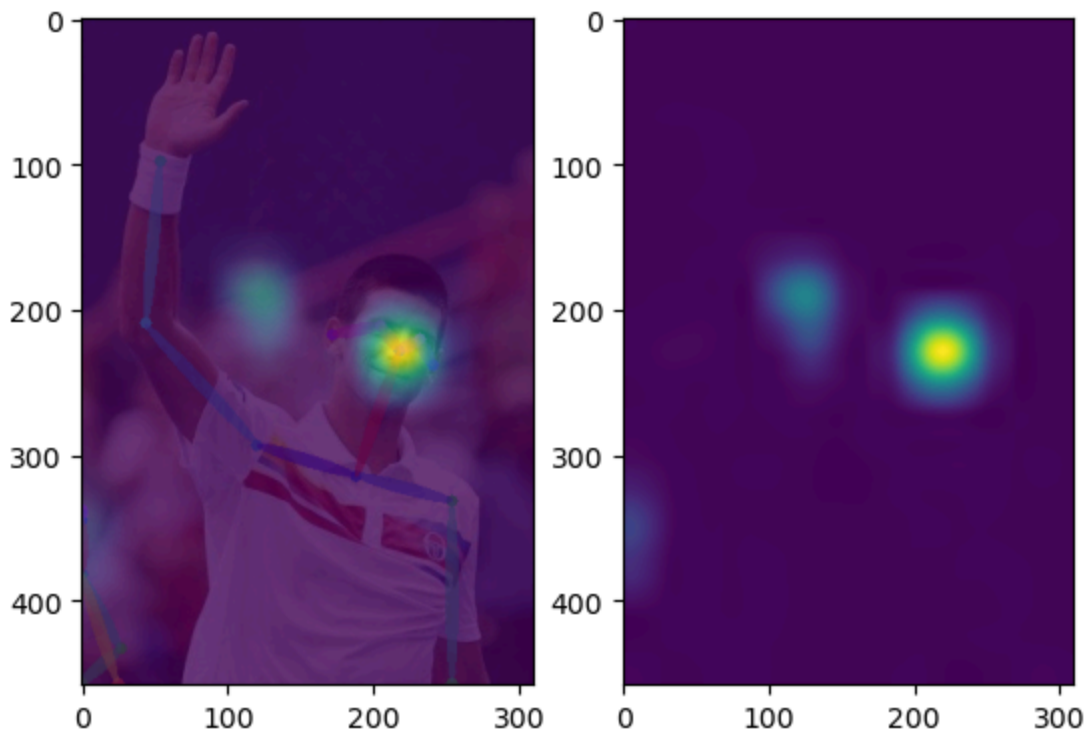
The pooling layer is responsible for this scaling, implemented as resize functions and squeeze functions

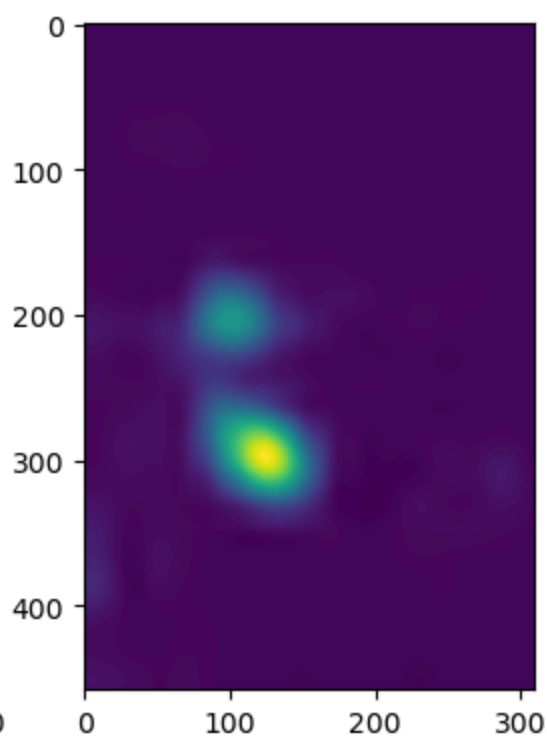
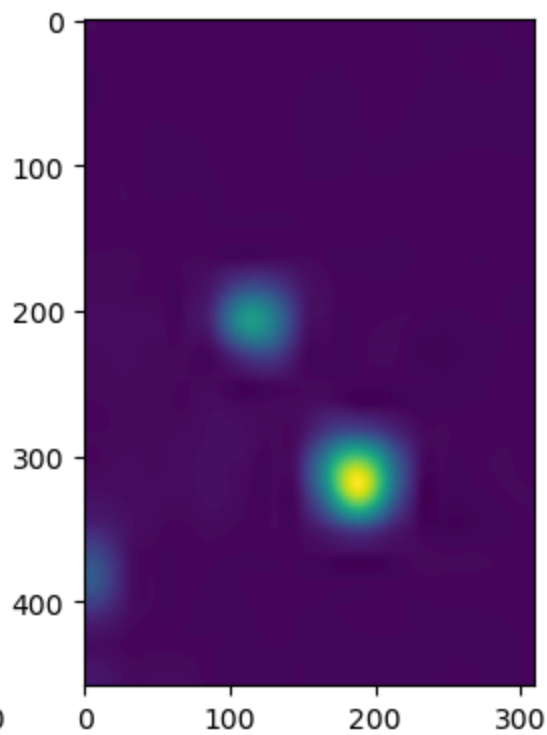
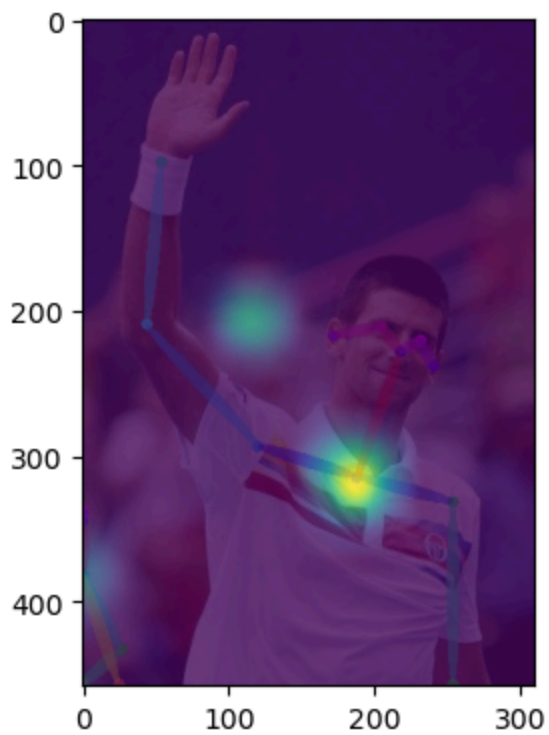
2. `heatmap_list_converted_list[0]` is a list of heatmaps from all 6 layers: [out1_2, out2_2, out3_2, out4_2, out5_2, out6_2], where the output of each layer is of shape [H, W, D].

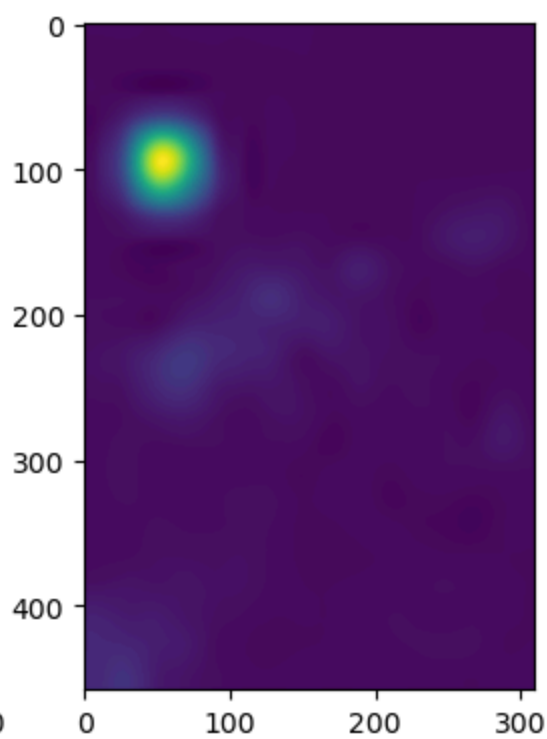
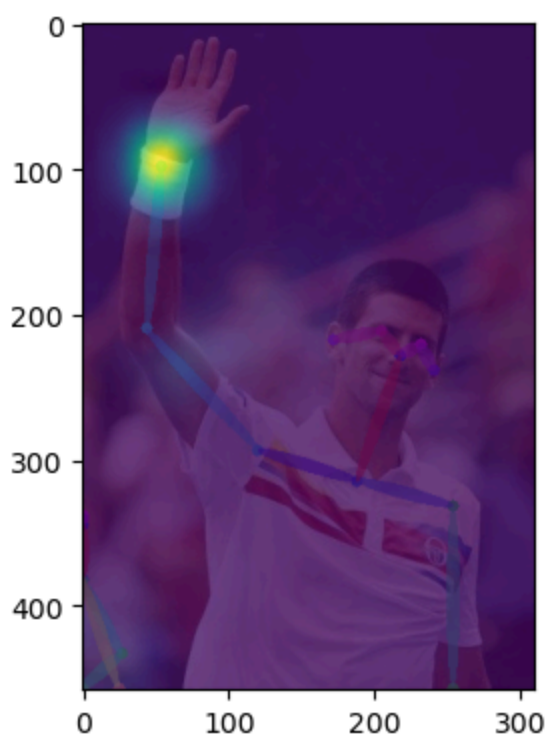
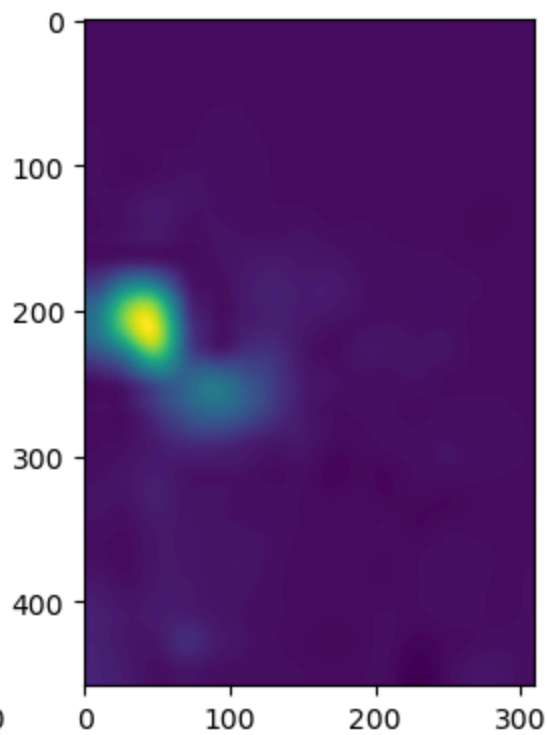
This snippet gives the function to visualize the d_{th} feature map from the layer **layer_idx**

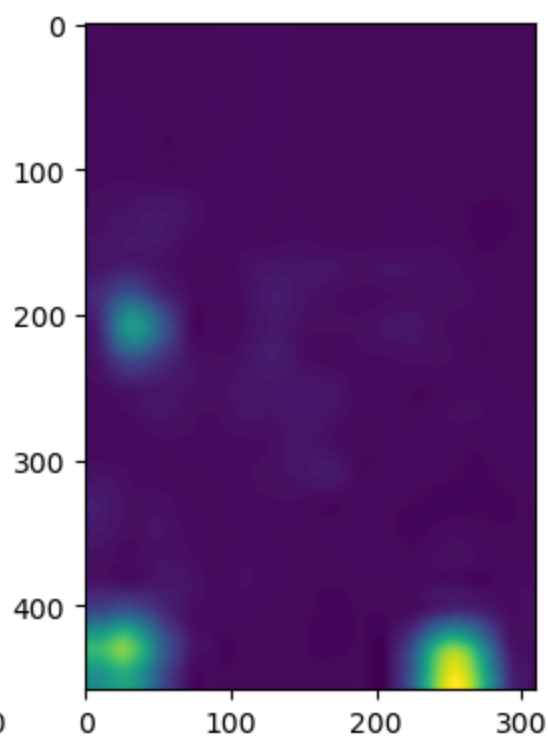
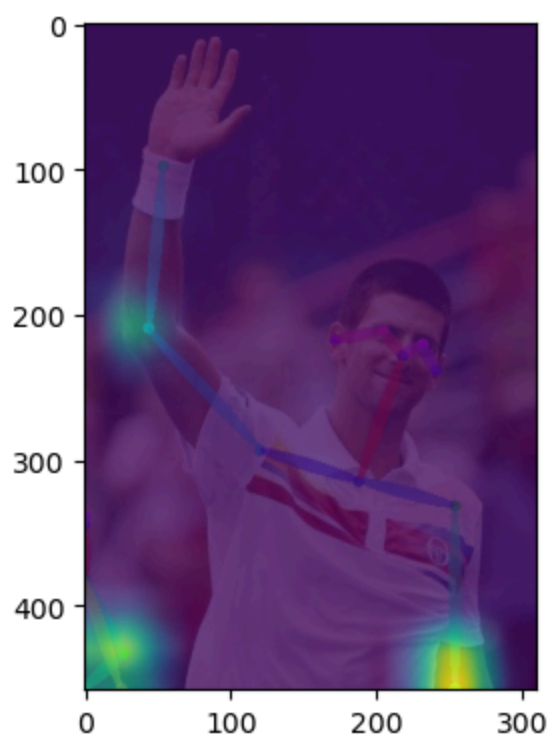
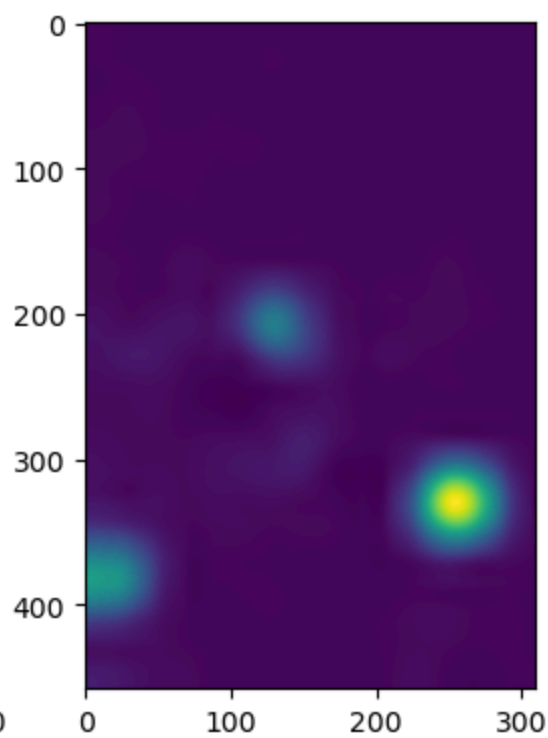
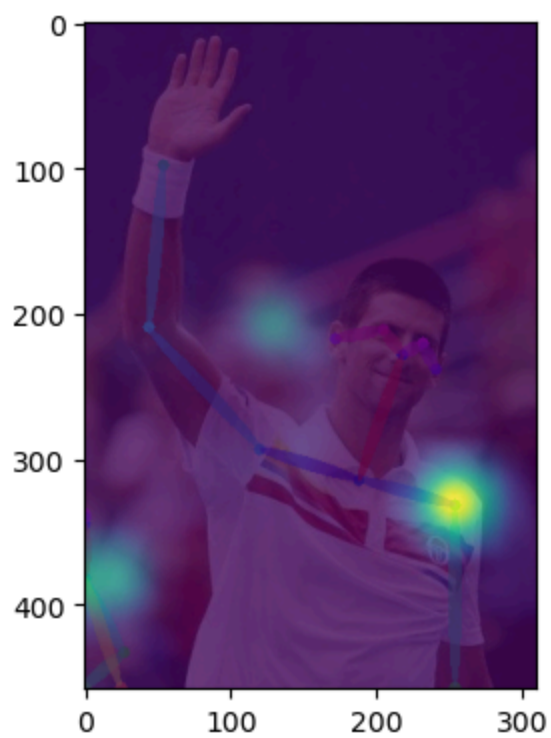
```
In [3]: layer_idx = -1

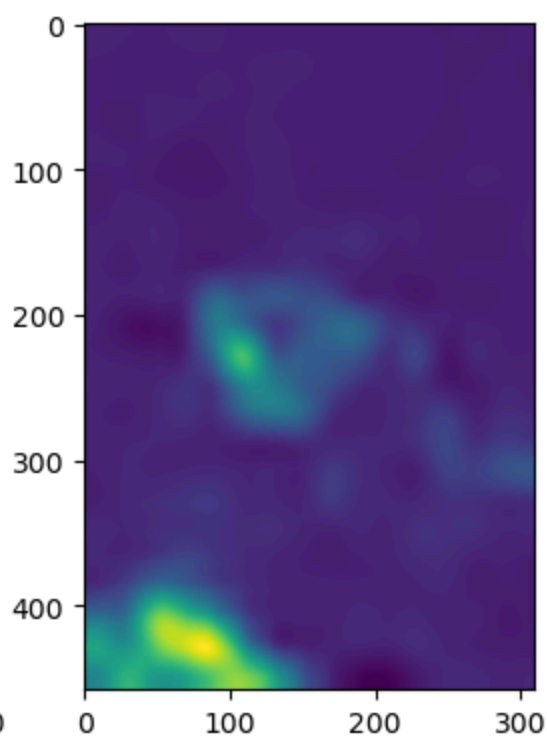
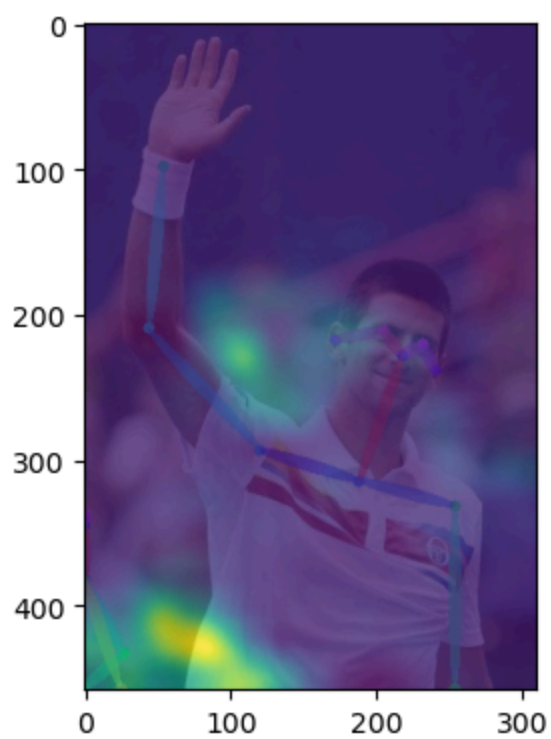
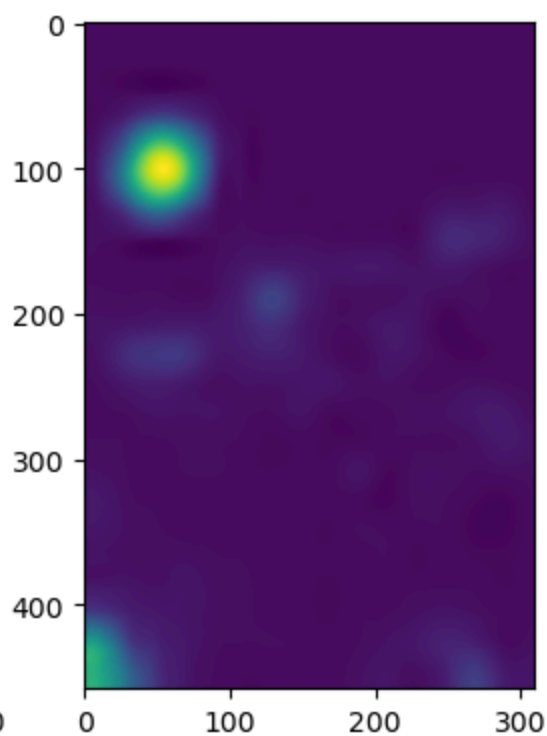
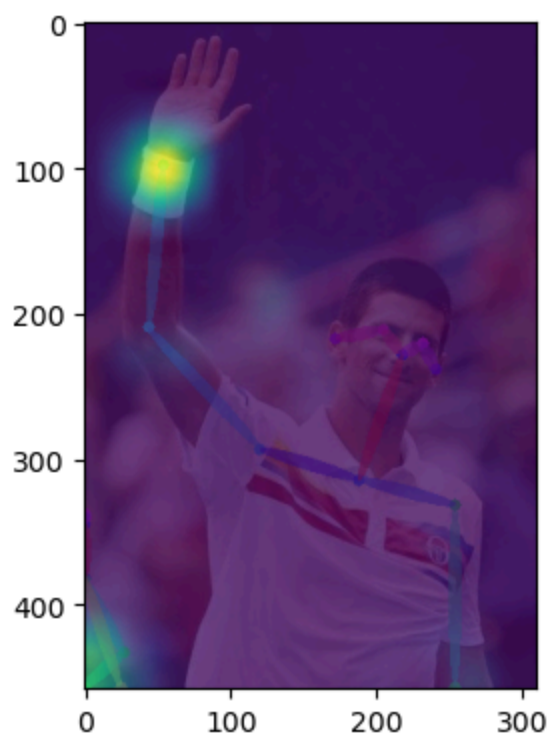
for d in range(heatmap_list_converted_list[0][layer_idx].shape[2]):
    layer_idx = 0
    heatmap = heatmap_list_converted_list[0][layer_idx][:, :, d]
    util.overlay_heatmap(canvas, heatmap)
```

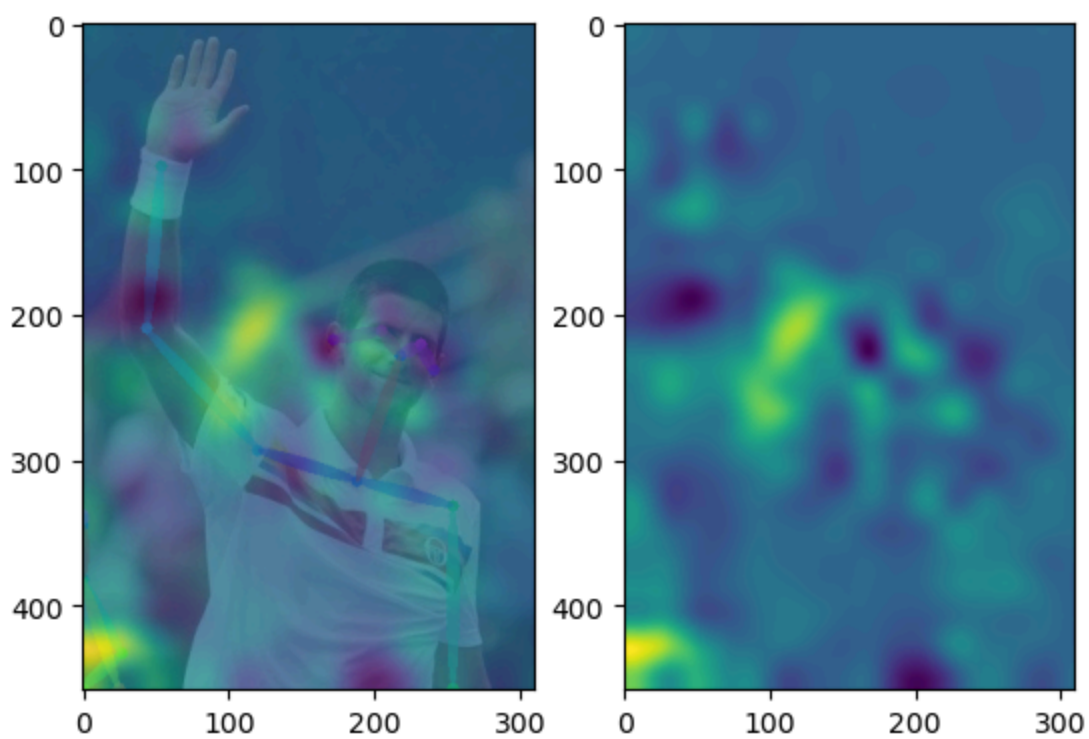
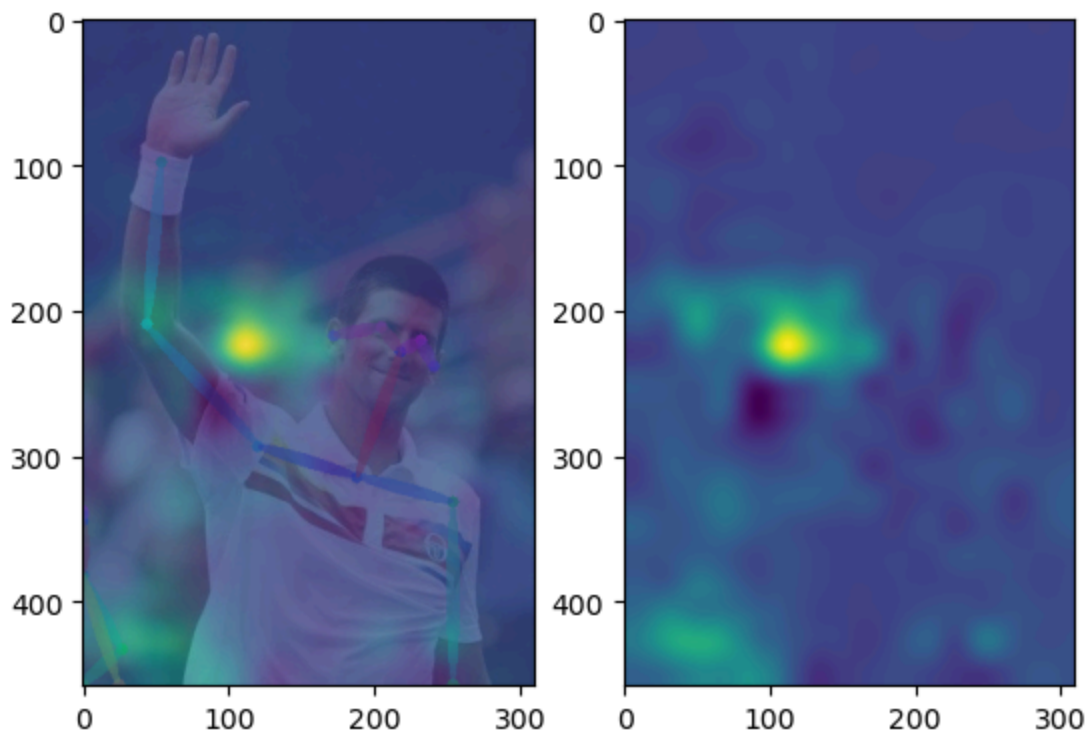


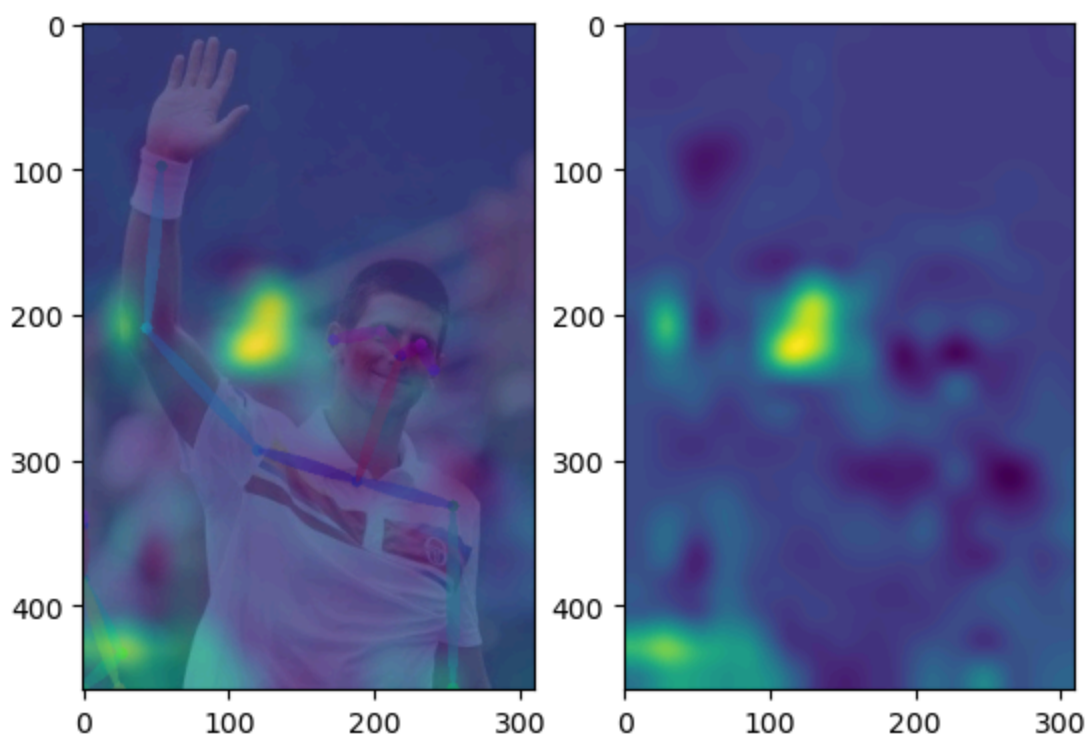
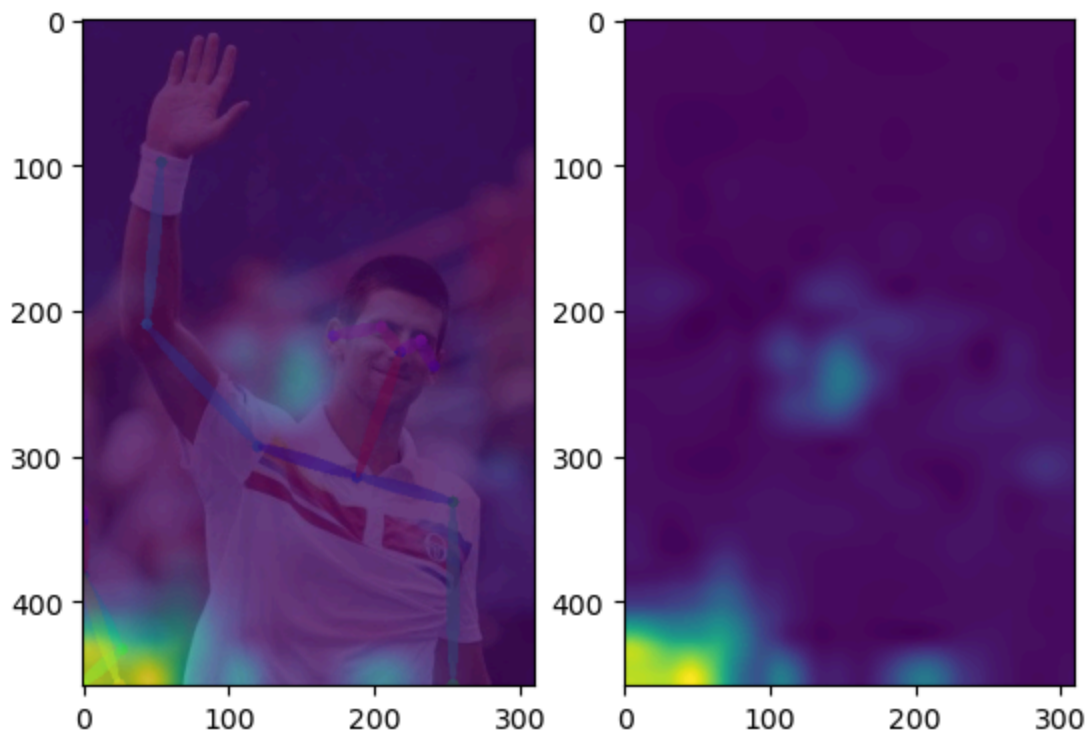


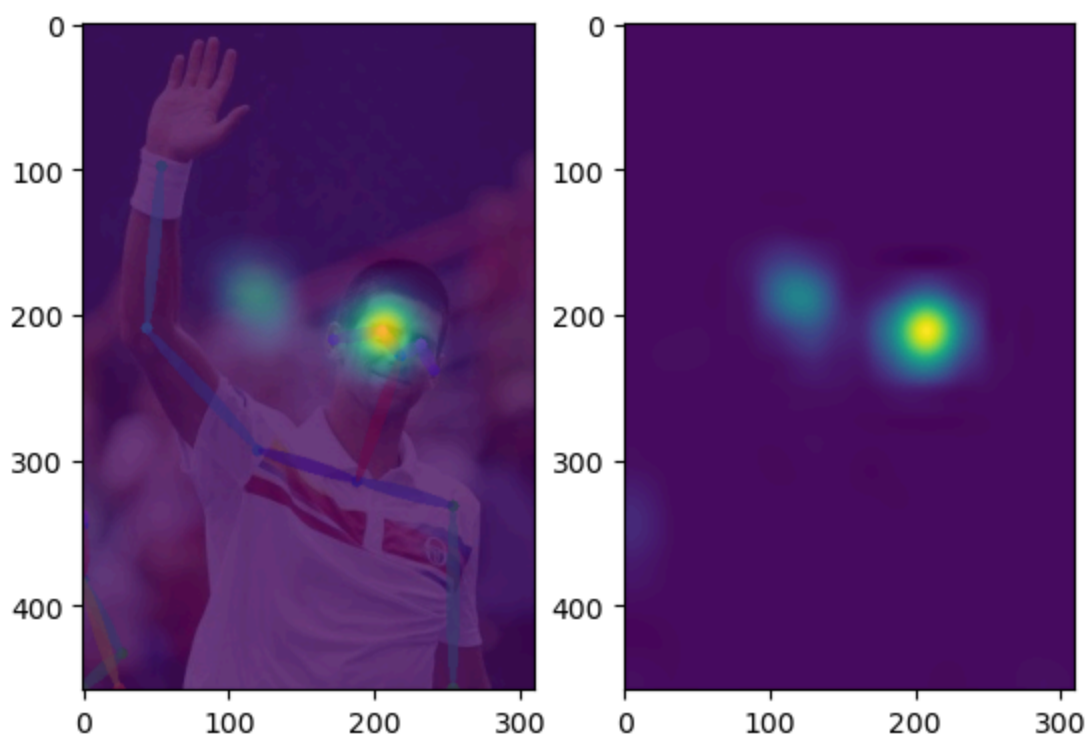
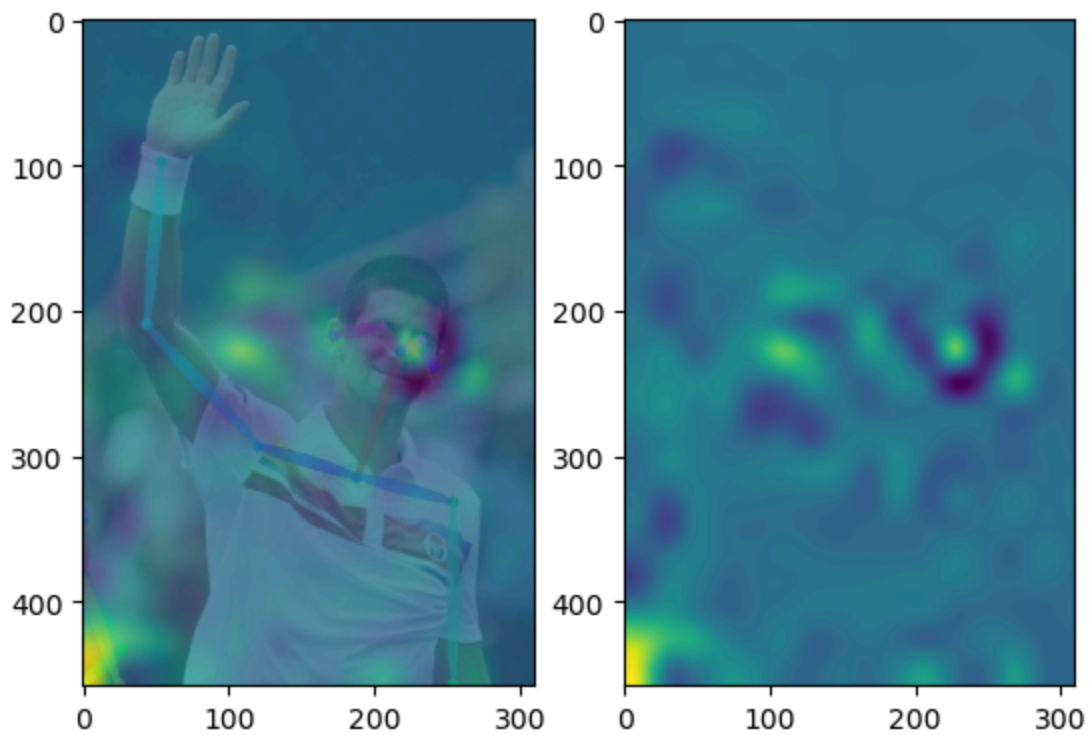


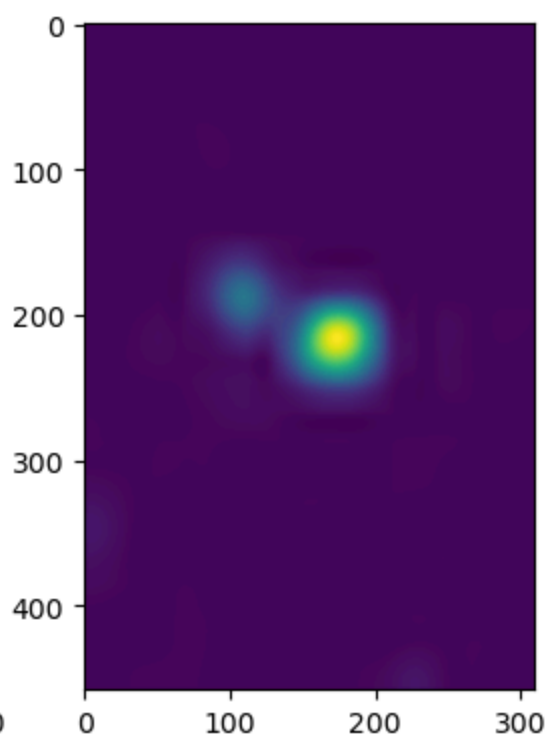
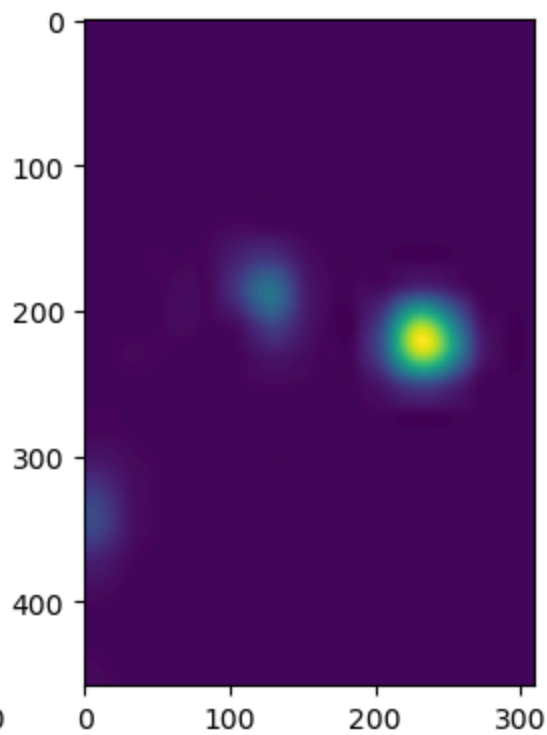


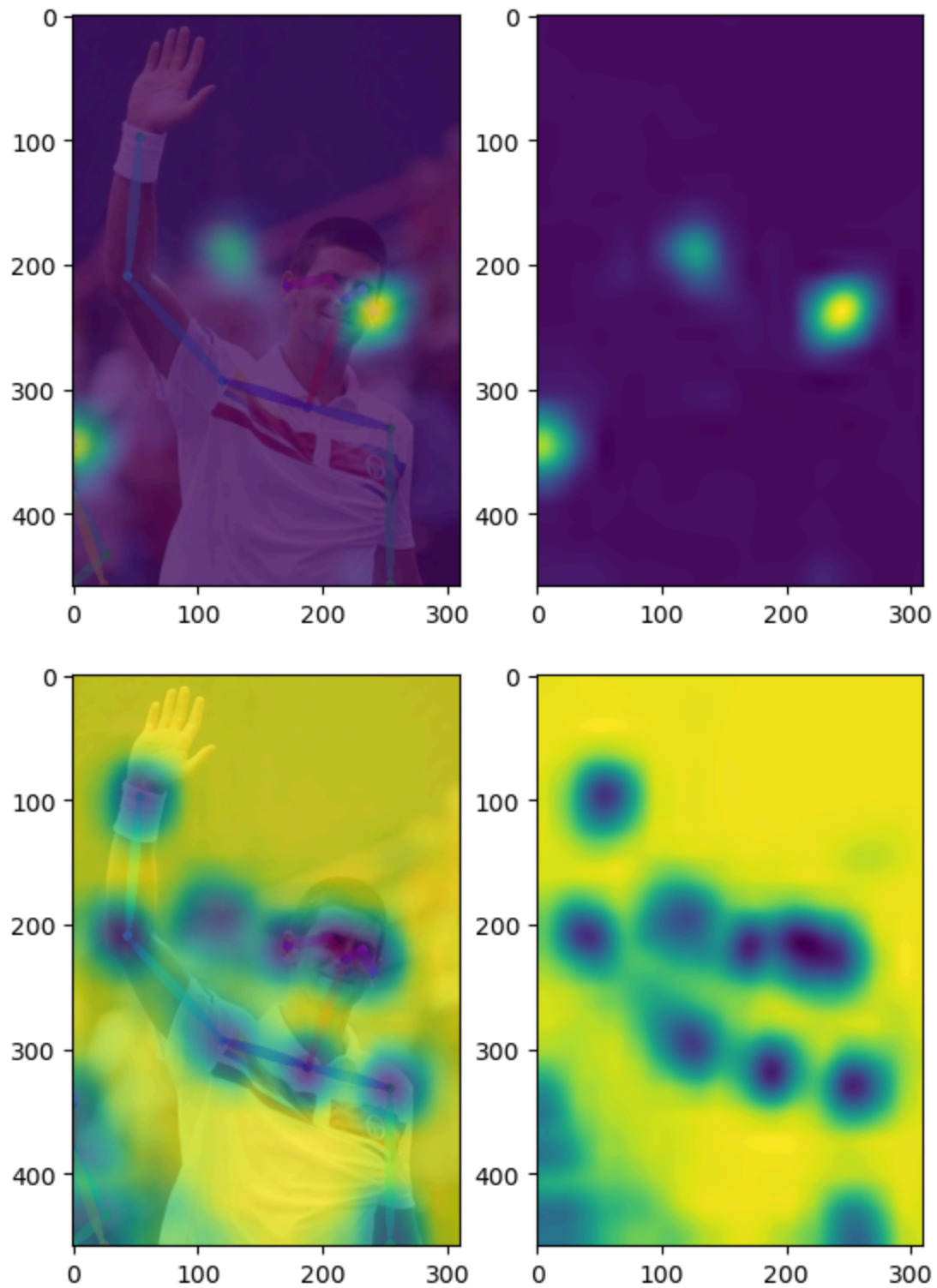










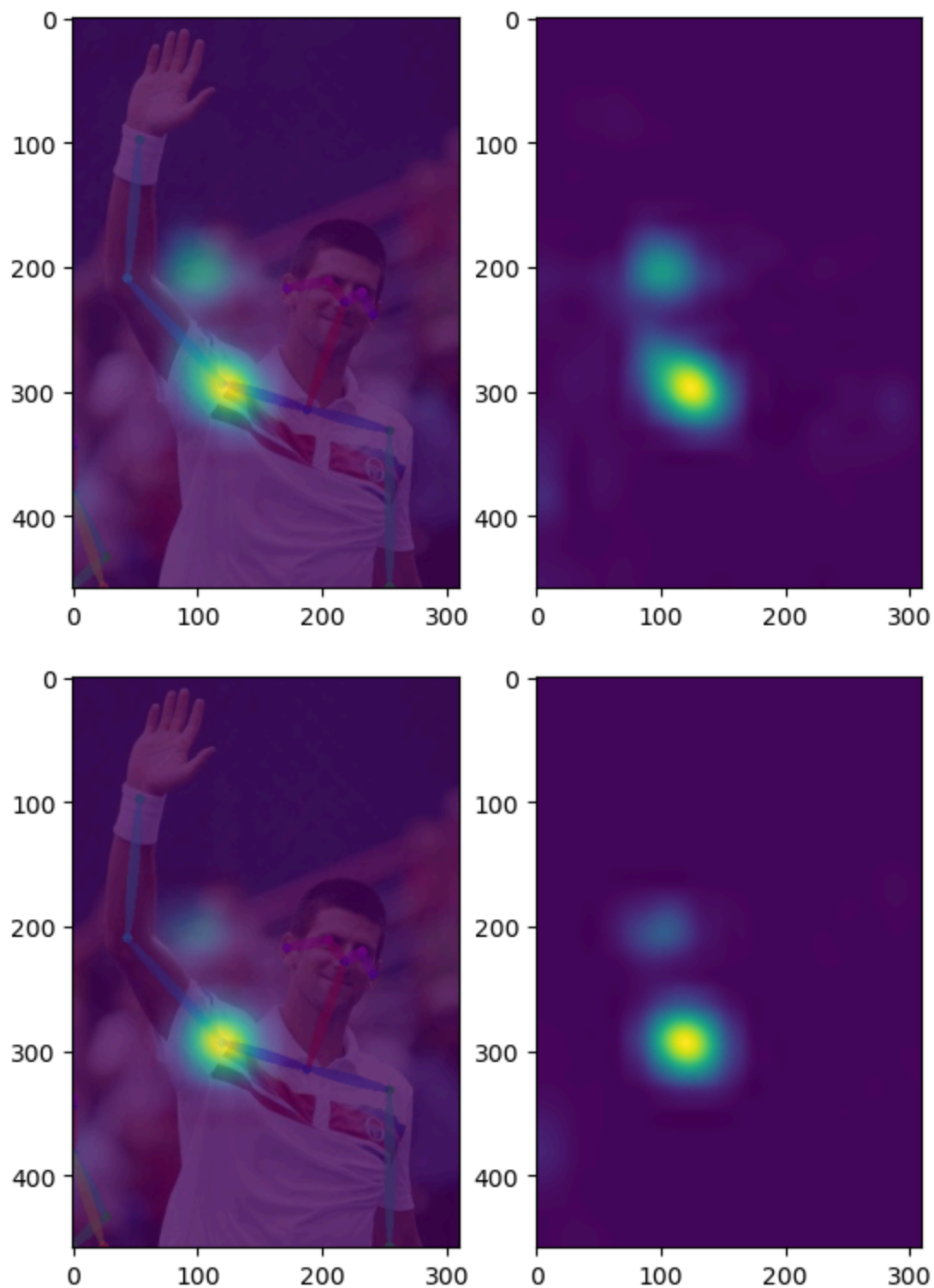


(a) What is D ? And given D can you tell what is the number of keypoints that the model is trying to estimate? **(5 points)**

D is 19, the model is trying to estimate 19 keypoints.

(b) Visualize and compare the heatmap from **layer_idx=0** and **layer_idx=5** for the keypoint **d=2**. **(5 points)**

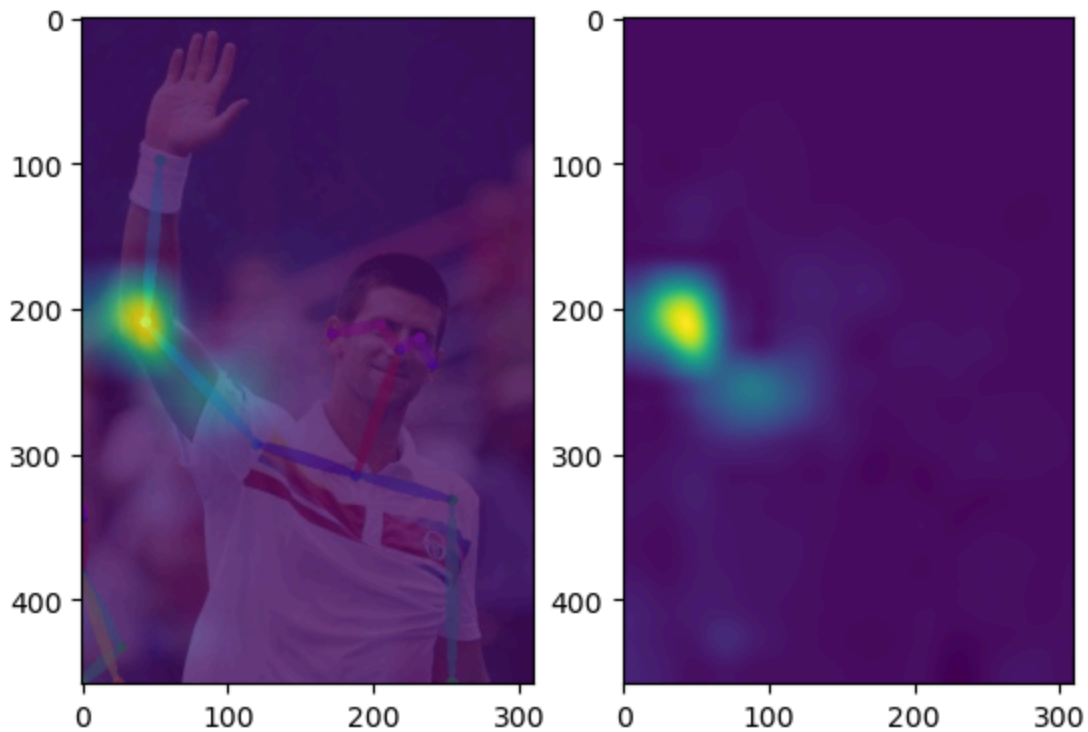

```
In [4]: layer_idx = 0
heatmap = heatmap_list_converted_list[0][layer_idx][:, :, 2]
util.overlay_heatmap(canvas, heatmap)
layer_idx = 5
heatmap = heatmap_list_converted_list[0][layer_idx][:, :, 2]
util.overlay_heatmap(canvas, heatmap)
```

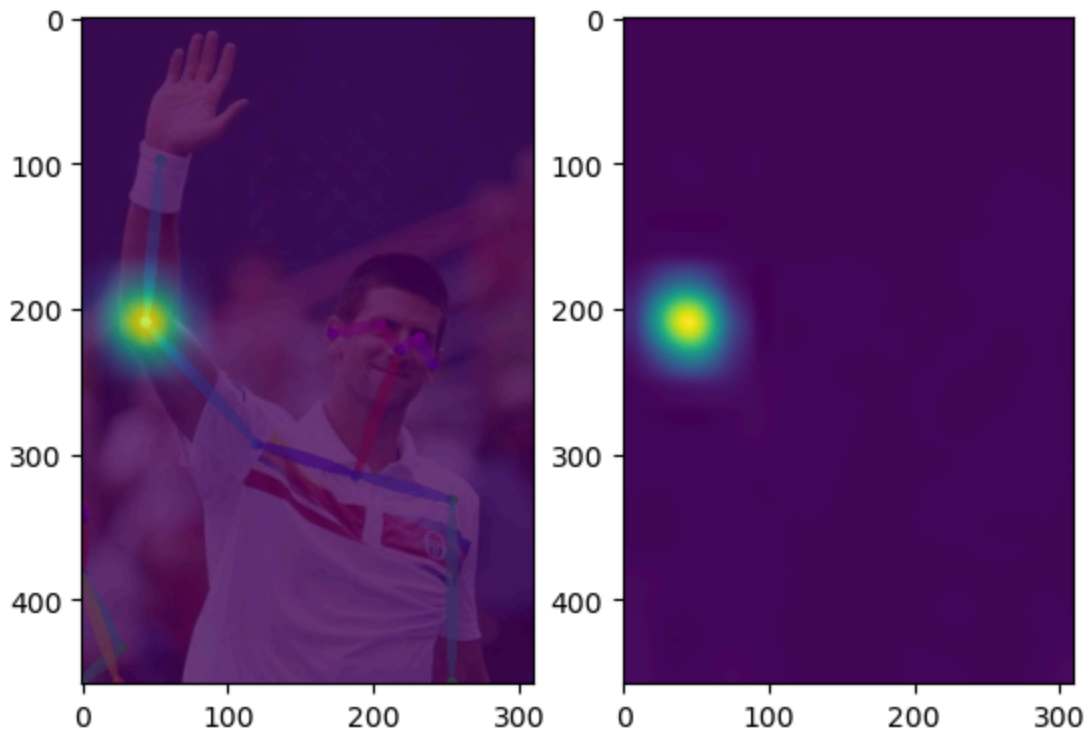


The heatmaps look very similar but the index 5 heatmap looks more localized to the keypoint and reduces the second recognized points

(c) Visualize and compare the heatmap from **layer_idx=0** and **layer_idx=5** for the keypoint **d=3**. (5 points)

```
In [5]: layer_idx = 0
heatmap = heatmap_list_converted_list[0][layer_idx][:, :, 3]
util.overlay_heatmap(canvas, heatmap)
layer_idx = 5
heatmap = heatmap_list_converted_list[0][layer_idx][:, :, 3]
util.overlay_heatmap(canvas, heatmap)
```





In []: Again, the main difference here **is** the heatmap becomes more localized **and** recudes n

(d) State and justify the shared difference in these comparisons? You may get some hint from the model design. **(5 points)**

The shared difference was the localization and noise reduction effects as the points got passed through the model. This makes sense when looking through the convolutional layers that we use to different degrees, as in the larger the layer index the further it goes through these convolutions and computations.

3. Set **multi_scale=True** in Line 38 of `body.py` and re-run the code from Q4(1).

(a) Visualize the output below. **(5 points)**

NOTE: you might need to restart the kernel each time you change the `multi_scale` flag. This may be required to re-initialize the model. [Kernel]--> [Restart]

```
In [2]: candidate, subset, heatmap_list, heatmap_list_converted_list = body_estimation(oriI
heatmap_0 = heatmap_list[0]
print(f"Heatmap shape: {heatmap_0.shape}")
canvas = copy.deepcopy(oriImg)
canvas = util.draw_bodypose(canvas, candidate, subset)

plt.figure(figsize=(10, 10))
plt.imshow(canvas[:, :,
                [2, 1, 0]])
```

```
# plt.axis('off')  
plt.show()
```

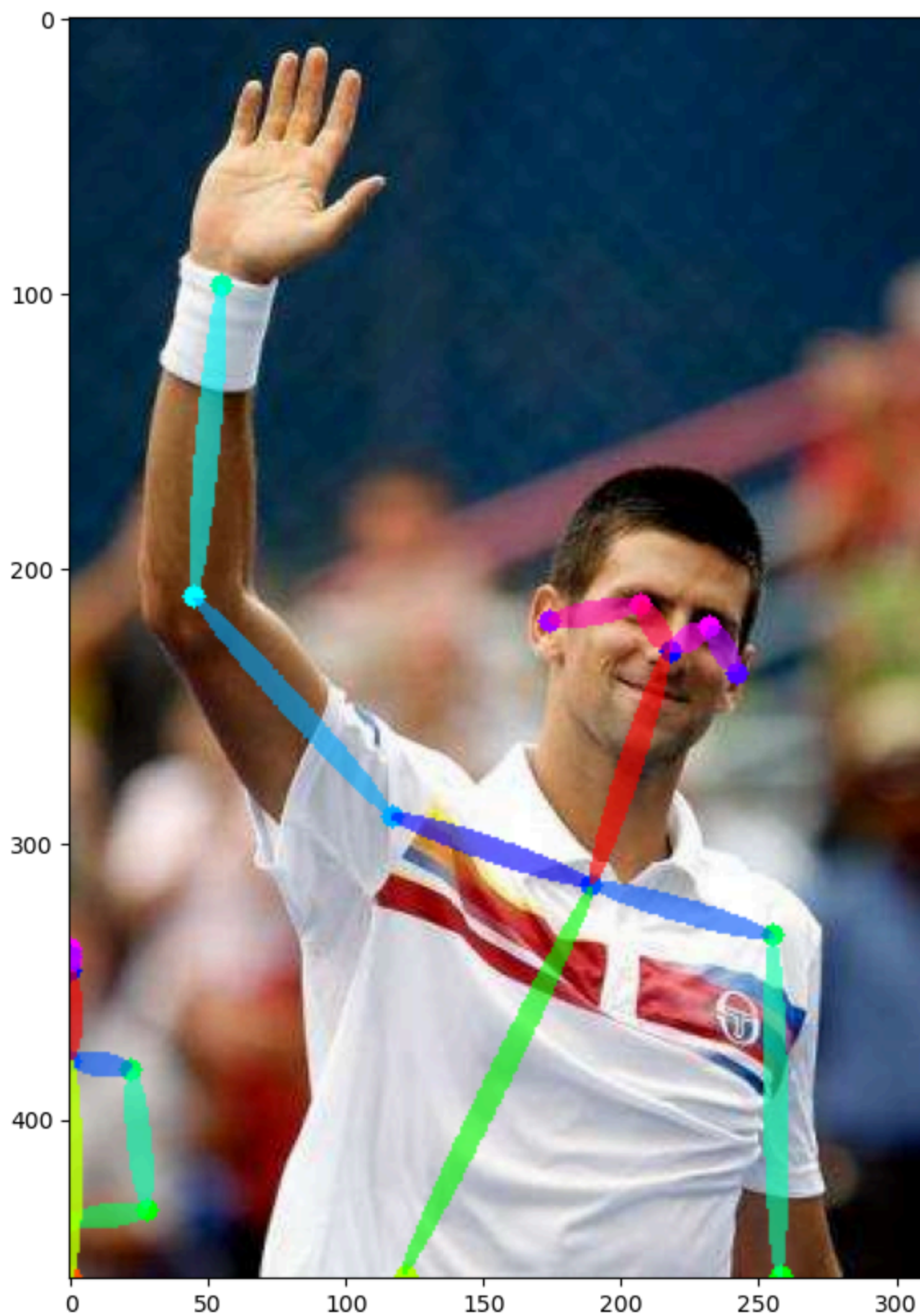
Original image shape: (458, 310, 3), New image shape: (1, 3, 184, 128)

Original image shape: (458, 310, 3), New image shape: (1, 3, 368, 256)

Original image shape: (458, 310, 3), New image shape: (1, 3, 552, 376)

Original image shape: (458, 310, 3), New image shape: (1, 3, 736, 504)

Heatmap shape: torch.Size([1, 19, 92, 63])



(b) Do the same comparison with additional images of `demo_coco.jpg`, `demo_coco2.jpg`, `demo_beach.jpg`, `demo_ucsd.jpg`, `demo_ucsd2.jpg`. (5 points)

```
In [5]: test_images = ['pytorch-openpose/images/demo_coco.jpg', 'pytorch-openpose/images/de
        'pytorch-openpose/images/demo_ucsd.jpg', 'pytorch-openpose/images/de
for test_image in test_images:
    oriImg = cv2.imread(test_image) # B,G,R order
    candidate, subset, heatmap_list, heatmap_list_converted_list = body_estimation(
    heatmap_0 = heatmap_list[0]
    print(f"Heatmap shape: {heatmap_0.shape}")
    canvas = copy.deepcopy(oriImg)
    canvas = util.draw_bodypose(canvas, candidate, subset)

    plt.figure(figsize=(10, 10))
    plt.imshow(canvas[:, :,
                    [2, 1, 0]])
    # plt.axis('off')
    plt.show()
## MULTISCALE TRUE
```

Original image shape: (425, 640, 3), New image shape: (1, 3, 184, 280)
 Original image shape: (425, 640, 3), New image shape: (1, 3, 368, 560)
 Original image shape: (425, 640, 3), New image shape: (1, 3, 552, 832)
 Original image shape: (425, 640, 3), New image shape: (1, 3, 736, 1112)
 Heatmap shape: torch.Size([1, 19, 92, 139])



Original image shape: (360, 640, 3), New image shape: (1, 3, 184, 328)
 Original image shape: (360, 640, 3), New image shape: (1, 3, 368, 656)
 Original image shape: (360, 640, 3), New image shape: (1, 3, 552, 984)
 Original image shape: (360, 640, 3), New image shape: (1, 3, 736, 1312)
 Heatmap shape: torch.Size([1, 19, 92, 164])



Original image shape: (480, 640, 3), New image shape: (1, 3, 184, 248)

Original image shape: (480, 640, 3), New image shape: (1, 3, 368, 496)

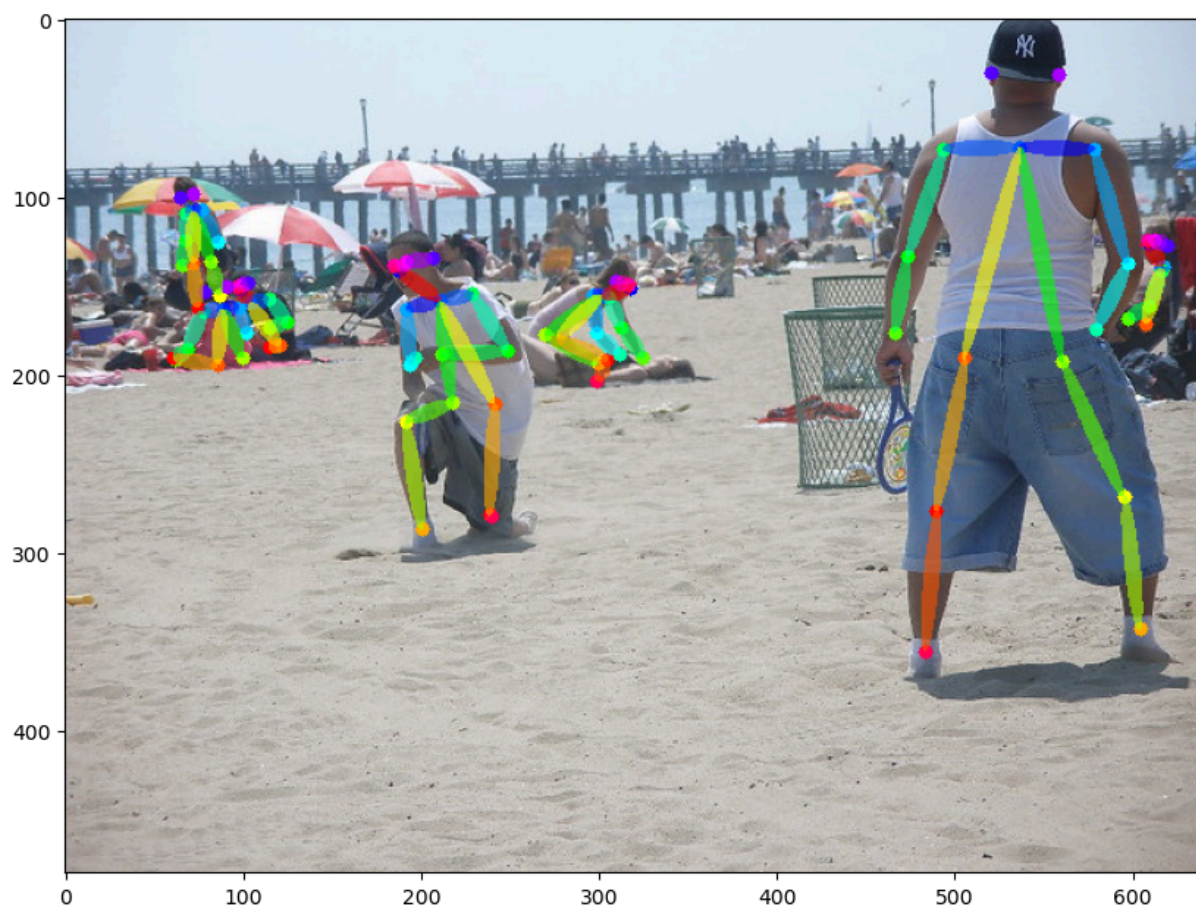
Original image shape: (480, 640, 3), New image shape: (1, 3, 552, 736)

Original image shape: (480, 640, 3), New image shape: (1, 3, 736, 984)

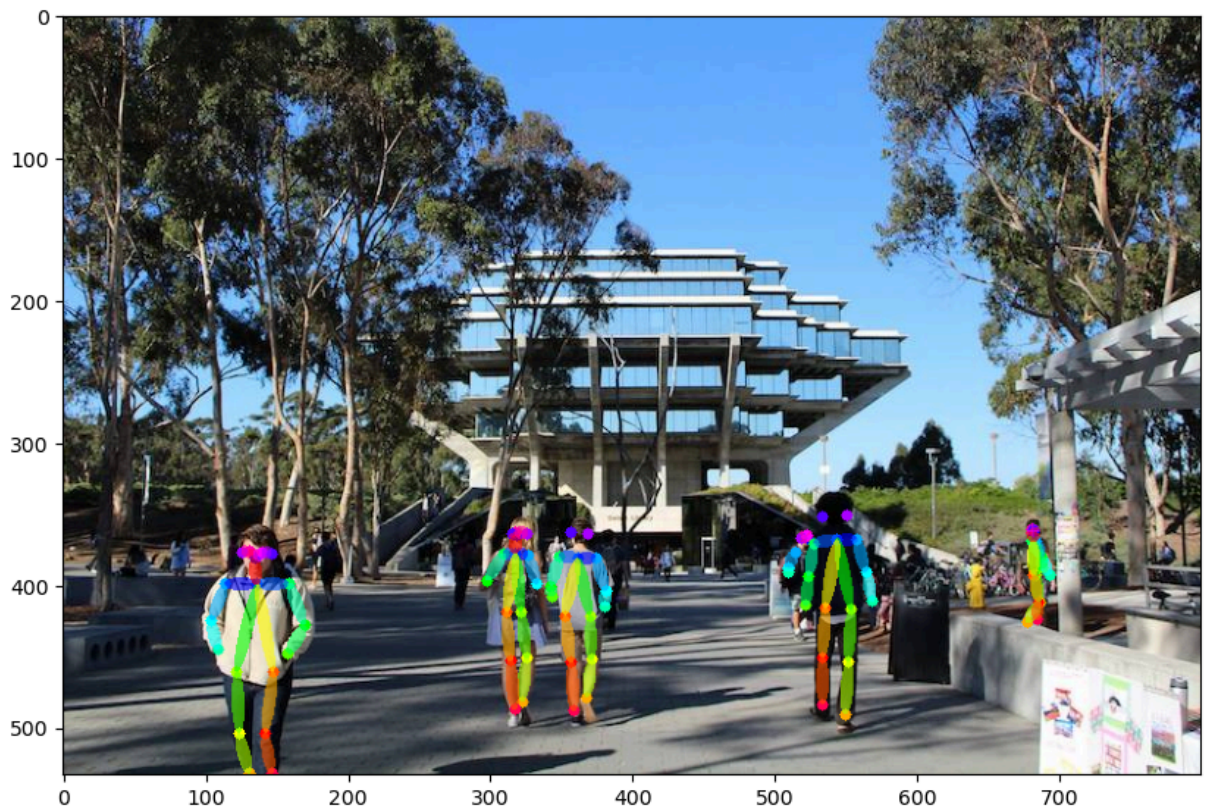
/home/dabrin/CSE152B-HW/HW3/pytorch-openpose/src/body.py:145: RuntimeWarning: invalid value encountered in divide

vec = np.divide(vec, norm)

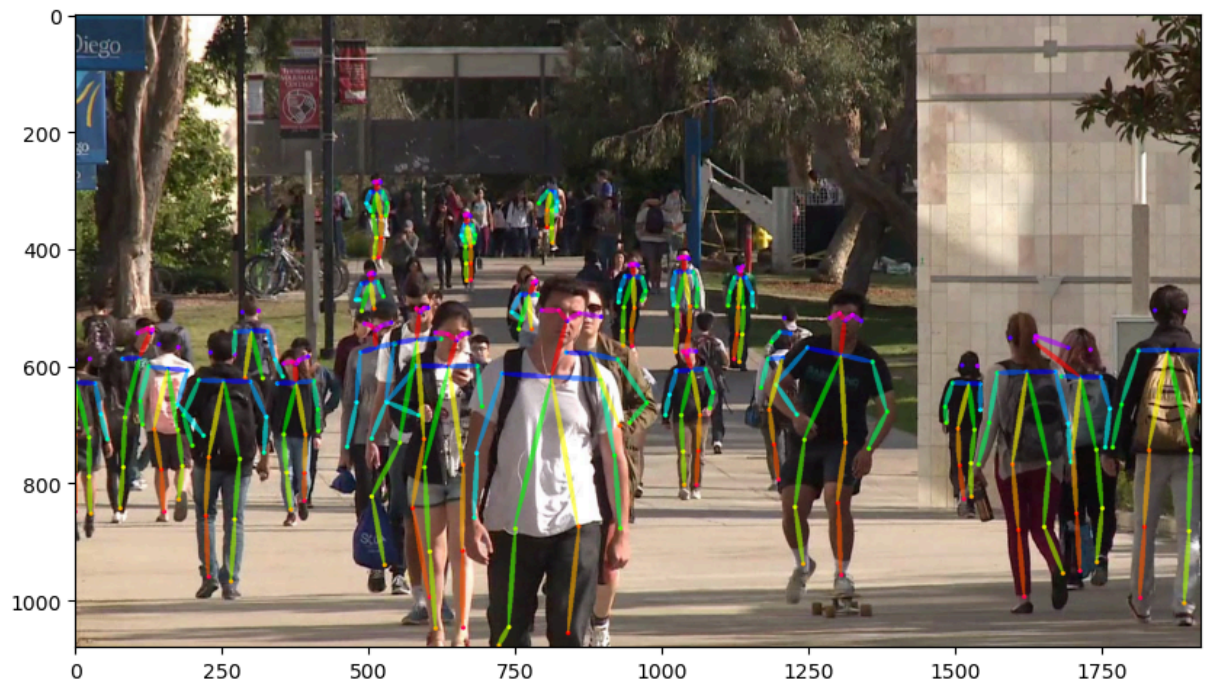
Heatmap shape: torch.Size([1, 19, 92, 123])



Original image shape: (533, 800, 3), New image shape: (1, 3, 184, 280)
Original image shape: (533, 800, 3), New image shape: (1, 3, 368, 552)
Original image shape: (533, 800, 3), New image shape: (1, 3, 552, 832)
Original image shape: (533, 800, 3), New image shape: (1, 3, 736, 1112)
Heatmap shape: torch.Size([1, 19, 92, 139])



Original image shape: (1080, 1920, 3), New image shape: (1, 3, 184, 328)
 Original image shape: (1080, 1920, 3), New image shape: (1, 3, 368, 656)
 Original image shape: (1080, 1920, 3), New image shape: (1, 3, 552, 984)
 Original image shape: (1080, 1920, 3), New image shape: (1, 3, 736, 1312)
 Heatmap shape: torch.Size([1, 19, 92, 164])



(c) What shared difference can you spot in most of the comparisons? Justify the difference.
(5 points)

The shared difference was the localization and noise reduction effects as the model gets more points or pixels. This makes sense when looking through the convolutional layers that we use to different degrees, as we can see more fine details and sharper lines as the original image gets larger.

Reference

- [1] Cao, Zhe. et al. "Realtime multi-person 2d pose estimation using part affinity fields." Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2017.
- [2] Wei, Shih-En. et al. "Convolutional pose machines." Proceedings of the IEEE conference on Computer Vision and Pattern Recognition. 2016.
- [3] Liu, Weiyang. et al. "SphereFace: Deep Hypersphere Embedding for Face Recognition." arXiv:1704.08063.
- [4] Huang, Gary. et al. "Labeled faces in the wild: A database for studying face recognition in unconstrained environments." Technical Report 07-49, University of Massachusetts, Amherst, October 2007.
- [5] Zhang, Kaipeng. et al. "Joint face detection and alignment using multitask cascaded convolutional networks." IEEE Signal Processing Letters, 23(10):1499–1503, 2016.
- [6] Yi, Dong. et al. Learning face representation from scratch. arXiv:1411.7923.
- [7] Wang, Hao. et al. Cosface: Large margin cosine loss for deep face recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pages 5265–5274, 2018.
- [8] Maaten, Laurens van der, and Geoffrey Hinton. "Visualizing data using t-SNE." Journal of machine learning research 9.Nov (2008): 2579-2605.